

# ITA0613

## Machine Learning

Name : Chiranjibi Pradhan  
Reg No : 192511475

### **Program 11: Credit Score Classification**

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
data = load_breast_cancer()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    test_size=0.3,
    random_state=42
)
```

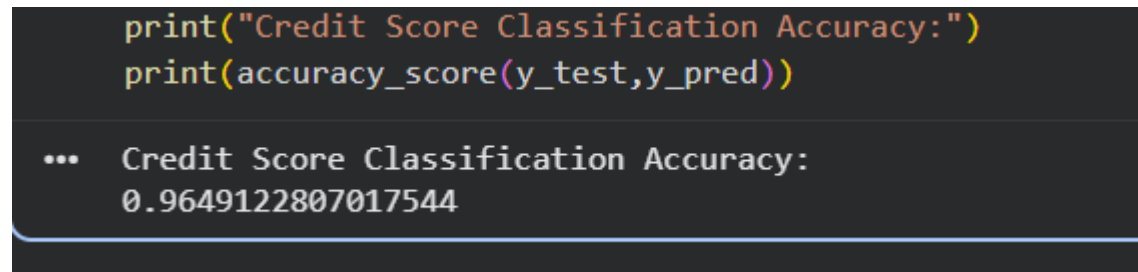
```
model = RandomForestClassifier()
```

```
model.fit(X_train,y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Credit Score Classification Accuracy:")  
print(accuracy_score(y_test,y_pred))
```

Output:



```
print("Credit Score Classification Accuracy:")  
print(accuracy_score(y_test,y_pred))  
  
... Credit Score Classification Accuracy:  
0.9649122807017544
```

### **Program 12: Iris Flower Classification using KNN**

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    iris.data,  
    iris.target,  
    test_size=0.3,  
    random_state=42  
)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

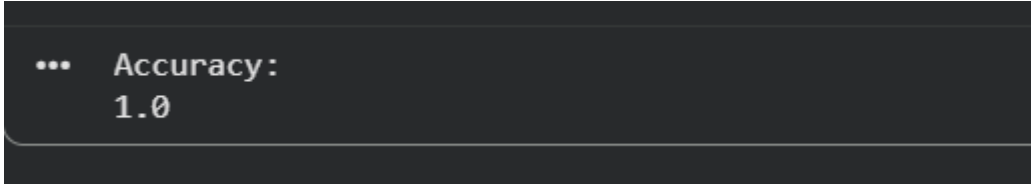
```
knn.fit(X_train,y_train)
```

```
y_pred = knn.predict(X_test)
```

```
print("Accuracy:")
```

```
print(accuracy_score(y_test,y_pred))
```

OUTPUT:

A screenshot of a terminal window with a dark background. It shows the output of a Python script: three dots followed by 'Accuracy:' on one line, and '1.0' on the line below.

```
... Accuracy:  
1.0
```

### **Program 13: Car Price Prediction**

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
data = pd.DataFrame({  
    'Year':[2018,2019,2020,2021,2022],  
    'Price':[500000,600000,700000,800000,900000]  
})
```

```
X = data[['Year']]
```

```
y = data['Price']
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
pred = model.predict([[2023]])
```

```
print("Predicted Car Price:")
```

```
print(pred[0])
```

OUTPUT:

```
.. Predicted Car Price:
   1000000.0
```

### **Program 14: House Price Prediction**

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
data = pd.DataFrame({
    'Area':[1000,1200,1500,1800,2000],
    'Price':[30,40,50,60,70]
})
```

```
X = data[['Area']]
```

```
y = data['Price']
```

```
model = LinearRegression()
```

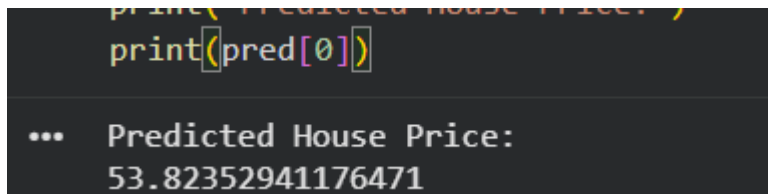
```
model.fit(X,y)
```

```
pred = model.predict([[1600]])
```

```
print("Predicted House Price:")
```

```
print(pred[0])
```

OUTPUT:

A screenshot of a Jupyter Notebook cell. The top part shows the code being executed: `print("Predicted House Price:")` and `print(pred[0])`. The bottom part shows the output: `... Predicted House Price:` followed by the value `53.82352941176471` on the next line.

```
print("Predicted House Price:")  
print(pred[0])  
  
... Predicted House Price:  
53.82352941176471
```

### **Program 15: Iris Flower Classification using Naive Bayes**

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
X_train,X_test,y_train,y_test = train_test_split(
```

```
    iris.data,
```

```
    iris.target,
```

```
    test_size=0.3,
```

```
    random_state=42
```

```
)
```

```
model = GaussianNB()
```

```
model.fit(X_train,y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:")
```

```
print(accuracy_score(y_test,y_pred))
```

OUTPUT:

```
print("Accuracy:")
print(accuracy_score(y_test,y_pred))

... Accuracy:
0.9777777777777777
```

### **Program 16: Compare Different Classification Algorithms**

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

```
iris = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```

iris.data,
iris.target,
test_size=0.3,
random_state=42
)

models = {
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(),
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(max_iter=200)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))

```

OUTPUT:

```

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))

... KNN Accuracy: 1.0
    Decision Tree Accuracy: 1.0
    Naive Bayes Accuracy: 0.9777777777777777
    Logistic Regression Accuracy: 1.0

```

## Program 17: Mobile Price Prediction

```
import pandas as pd

from sklearn.ensemble import RandomForestRegressor

data = pd.DataFrame({
    'RAM':[2,4,6,8,12],
    'Storage':[32,64,128,128,256],
    'Price':[10000,15000,20000,25000,40000]
})

X = data[['RAM','Storage']]
y = data['Price']

model = RandomForestRegressor(random_state=42)

model.fit(X,y)

prediction = model.predict([[8,128]])

print("Predicted Mobile Price:")
print(prediction[0])
```

OUTPUT:

```
print("Predicted Mobile Price:")
print(prediction[0])

.. Predicted Mobile Price:
22700.0
```



### **Program 18: Perceptron Based IRIS Classification**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=42
)

model = Perceptron()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:")
print(accuracy_score(y_test, y_pred))
```

OUTPUT:

```
print("Accuracy:")
print(accuracy_score(y_test, y_pred))

.. Accuracy:
0.8222222222222222
```

### Program 19: Bank Loan Prediction using Naive Bayes

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
```

```
data = pd.DataFrame({
    'Income':[20,30,40,50,60],
    'Age':[25,35,45,30,50],
    'Loan':[0,0,1,1,1]
})
```

```
X = data[['Income','Age']]
y = data['Loan']
```

```
model = GaussianNB()
```

```
model.fit(X,y)
```

```
prediction = model.predict([[55,40]])
```

```
print("Loan Prediction:")
```

```
print("Approved" if prediction[0] == 1 else "Rejected")
```

OUTPUT:

```
print("Loan Prediction:")
print("Approved" if prediction[0] == 1 else "Rejected")

... Loan Prediction:
Approved
```

## Program 20: Future Sales Prediction

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
data = pd.DataFrame({
    'Month':[1,2,3,4,5,6,7,8,9,10],
    'Sales':[100,120,130,150,170,180,200,210,230,250]
})
```

```
X = data[['Month']]
```

```
y = data['Sales']
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
future_month = [[12]]
```

```
prediction = model.predict(future_month)
```

```
print("Predicted Sales for Month 12:")
```

```
print(prediction[0])
```

OUTPUT:

```
print("Predicted Sales for Month 12:")  
print(prediction[0])
```

```
... Predicted Sales for Month 12:  
279.57575757575756
```